Experiment No. (4):

Write a JAVA program that implements a multi-thread application that has three threads. First thread generates Random integer every 1 second and if the value is even, Second thread computes the square of the number and prints. If the value is Odd, the third will print the value of cube of the number.

```
package recordprogs;

import java.util.Random;

class Even implements Runnable
{
   public int x;

   public Even(int x)
   {
      this.x = x;
   }

   public void run()
   {
      System.out.println(
         "Thread Name: Even Thread and " + x
         + " is even number and Square of " + x
         + " is: " + (x * x)
      );
   }
}

class Odd implements Runnable
{
   public int x;

   public Odd(int x)
   {
      this.x = x;
   }

   public void run()
   {
      System.out.println(
         "Thread Name: Odd Thread and " + x
         + " is odd number and Cube of " + x
         + " is: " + (x * x * x)
      );
```

```
    }
}

class A extends Thread
{
    public String tname;
    public Random r;
    public Thread t1, t2;

    public A(String s)
    {
        tname = s;
    }

    public void run()
    {
        int num = 0;
        r = new Random();

        try
        {
            for (int i = 0; i < 50; i++)
            {
                num = r.nextInt(100);

                System.out.println(
                    "Main Thread and Generated Number is: " + num
                );

                if (num % 2 == 0)
                {
                    t1 = new Thread(new Even(num));
                    t1.start();
                }
                else
                {
                    t2 = new Thread(new Odd(num));
                    t2.start();
                }

                Thread.sleep(1000);

                System.out.println("-----------------------------");
            }
        }
        catch (InterruptedException ex)
```

```
        {
            System.out.println("Thread interrupted: " + ex.getMessage());
            Thread.currentThread().interrupt();
        }
    }
}

public class Multithreading
{
    public static void main(String[] args)
    {
        A a = new A("One");
        a.start();
    }
}
```

Output:

```
Main Thread and Generated Number is 34
Thread Name:Even Thread and 34is even Number and Square of 34is: 1156
-----------------------------------------------------------------------------
Main Thread and Generated Number is 44
Thread Name:Even Thread and 44is even Number and Square of 44is: 1936
-----------------------------------------------------------------------------
Main Thread and Generated Number is 47
Thread Name:ODD Thread and 47 is odd number and Cube of 47 is:103823
-----------------------------------------------------------------------------
Main Thread and Generated Number is 25
Thread Name:ODD Thread and 25 is odd number and Cube of 25 is:15625
-----------------------------------------------------------------------------
Main Thread and Generated Number is 93
Thread Name:ODD Thread and 93 is odd number and Cube of 93 is:804357
-----------------------------------------------------------------------------
Main Thread and Generated Number is 36
Thread Name:Even Thread and 36is even Number and Square of 36is: 1296
-----------------------------------------------------------------------------
Main Thread and Generated Number is 70
Thread Name:Even Thread and 70is even Number and Square of 70is: 4900
```

Viva Questions:

1. **What is Multithreading?**
   Multithreading in java is a process of executing multiple threads simultaneously.

   Thread is basically a lightweight sub-process, a smallest unit of processing. Multiprocessing and multithreading, both are used to achieve multitasking.

   But we use multithreading than multiprocessing because threads share a common memory area. They don't allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.

   Java Multithreading is mostly used in games, animation etc.

2. What are the Advantages of Java Multithreading?

1) It doesn't block the user because threads are independent and you can perform multiple operations at same time.
2) You can perform many operations together so it saves time.
3) Threads are independent so it doesn't affect other threads if exception occur in a single thread.

3. What is Multitasking? Differences between process and thread based multitasking?

Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU. Multitasking can be achieved by two ways:

1) Process-based Multitasking(Multiprocessing)
2) Thread-based Multitasking(Multithreading)

1) Process-based Multitasking (Multiprocessing)

- Each process have its own address in memory i.e. each process allocates separate memory area.
- Process is heavyweight.
- Cost of communication between the process is high.
- Switching from one process to another require some time for saving and loading registers, memory maps, updating lists etc.

2) Thread-based Multitasking (Multithreading)

- Threads share the same address space.
- Thread is lightweight.
- Cost of communication between the thread is low.

Thread:

A thread is a lightweight sub process, a smallest unit of processing. It is a separate path of execution.

Threads are independent, if there occurs exception in one thread, it doesn't affect other threads. It shares a common memory area.

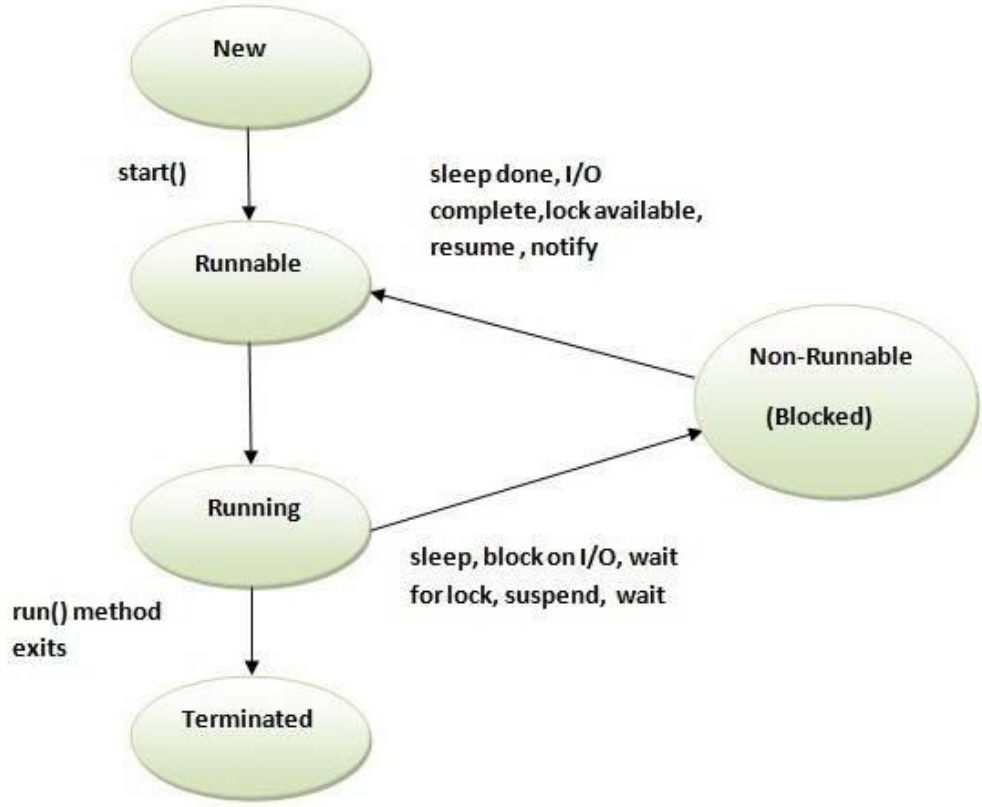
4. What is the Life cycle of a Thread (Thread States)?

A thread can be in one of the five states. According to sun, there is only 4 states in thread life cycle in java new, runnable, non-runnable and terminated. There is no running state.

But for better understanding the threads, we are explaining it in the 5 states.

The life cycle of the thread in java is controlled by JVM. The java thread states are as follows:

- New
- Runnable
- Running
- Non-Runnable (Blocked)
- Terminated



1) New

The thread is in new state if you create an instance of Thread class but before the invocation ofstart() method.

2) Runnable

The thread is in runnable state after invocation of start() method, but the thread scheduler hasnot selected it to be the running thread.

3) Running

The thread is in running state if the thread scheduler has selected it.

4) Non-Runnable (Blocked)

This is the state when the thread is still alive, but is currently not eligible to run.

5) Terminated

A thread is in terminated or dead state when its run() method exits.

5. How to create thread?

There are two ways to create a thread:

- By extending Thread class
- By implementing Runnable interface.

Thread class:

Thread class provide constructors and methods to create and perform operations on a thread.Thread class extends Object class and implements Runnable interface.

Commonly used Constructors of Thread class:

Thread()
Thread(Strin
g name)
Thread(Runn
able r)
Thread(Runnable r,String name)

Commonly used methods of Thread class:

1. **public void run():** is used to perform action for a thread.
2. **public void start():** starts the execution of the thread.JVM calls the run() method on thethread.
3. **public void sleep(long miliseconds):** Causes the currently executing thread to sleep(temporarily cease execution) for the specified number of milliseconds.
4. **public void join():** waits for a thread to die.
5. **public void join(long miliseconds):** waits for a thread to die for the specified miliseconds.
6. **public int getPriority():** returns the priority of the thread.
7. **public int setPriority(int priority):** changes the priority of the thread.
8. **public String getName():** returns the name of the thread.
9. **public void setName(String name):** changes the name of the thread.
10. **public Thread currentThread():** returns the reference of currently executing thread.
11. **public int getId():** returns the id of the thread.
12. **public Thread.State getState():** returns the state of the thread.
13. **public boolean isAlive():** tests if the thread is alive.
14. **public void yield():** causes the currently executing thread object to temporarily pause andallow other threads to execute.
15. **public void suspend():** is used to suspend the thread(depricated).
16. **public void resume():** is used to resume the suspended thread(depricated).
17. **public void stop():** is used to stop the thread(depricated).
18. **public boolean isDaemon():** tests if the thread is a daemon thread.
19. **public void setDaemon(boolean b):** marks the thread as daemon or user thread.
20. **public void interrupt():** interrupts the thread.
21. **public boolean isInterrupted():** tests if the thread has been interrupted.
22. **public static boolean interrupted():** tests if the current thread has been interrupted.

Runnable interface:

The Runnable interface should be implemented by any class whose instances are intended to beexecuted by a thread. Runnable interface have only one method named run().

1. **public void run():** is used to perform action for a thread.

Starting a thread:

start() method of Thread class is used to start a newly created thread. It performs following tasks:

- A new thread starts(with new callstack).
- The thread moves from New state to the Runnable state.
- When the thread gets a chance to execute, its target run() method will run.